3b-verifiable-ai-execution

Table of Contents

Requirements Document

Introduction

This document defines the requirements for the Verifiable AI Execution subsystem (Team 3b) of the Autonomous Web3 Research & On-Chain Settlement Agents platform. The subsystem manages off-chain AI inference execution while generating cryptographic proofs of correct execution. It integrates Trusted Execution Environments (TEE) for hardware-attested compute isolation, produces zero-knowledge Machine Learning (zkML) proofs for verifiable inference, and orchestrates federated learning pipelines using the Flower framework. The system listens to Team 3a's blockchain events (TaskCreated, TaskFunded) to trigger AI workloads and submits execution proofs (attestations) and Content Identifiers (CIDs) of model artifacts back to the blockchain for on-chain verification and settlement.

Glossary

- **Inference_Engine**: The Python-based FastAPI service that receives AI inference requests, loads models, executes inference within a TEE, and returns results with cryptographic proofs.
- **TEE_Runtime**: The Trusted Execution Environment runtime (Intel SGX or AMD SEV simulation) that provides hardware-isolated execution and memory encryption for AI model loading and inference.
- **Proof_Generator**: The component responsible for generating cryptographic proofs of correct AI inference execution, including TEE attestation reports and zkML proofs.
- **zkML_Prover**: The zero-knowledge proof subsystem that generates succinct proofs verifying that a specific ML model produced a specific output for a given input without revealing model weights.
- **Federated_Learning_Orchestrator**: The Flower-framework-based component that coordinates federated learning rounds across distributed participants, aggregates model updates, and manages training pipelines.
- **Event_Listener**: The component that monitors Team 3a's blockchain for TaskCreated and TaskFunded events and dispatches corresponding AI workloads to the Inference_Engine.

- **Proof_Submitter**: The component that submits execution proofs (TEE attestations and zkML proofs) and Content Identifiers (CIDs) of model artifacts to Team 3a's smart contracts on-chain.
- **Model_Registry**: The secure storage and versioning system for AI models loaded into the TEE_Runtime, backed by Redis for metadata and decentralized storage for model binaries.
- **Attestation_Report**: A cryptographic document generated by the TEE_Runtime that proves code integrity, data isolation, and correct execution within the hardware enclave.
- **CID**: A Content Identifier — a content-addressed hash referencing a model artifact stored on decentralized storage (IPFS/Filecoin/0G).
- **Task**: A unit of work posted on-chain by Team 3a's TaskManager contract, containing a task identifier, description hash, deadline, and bounty amount.
- **Execution_Receipt**: A structured record containing the inference result, associated proofs, model CID, and metadata produced after a successful AI inference execution.
- **Verification_Interface**: The API endpoint that allows external parties to verify submitted execution proofs without requiring TEE hardware.

Requirements

Requirement 1: Off-Chain AI Inference Execution

User Story: As a task requester, I want AI inference tasks to be executed off-chain with deterministic outputs, so that computational workloads are processed efficiently while maintaining verifiability.

Acceptance Criteria

1. WHEN the Inference_Engine receives a valid inference request containing a model identifier and input data, THE Inference_Engine SHALL load the specified model and execute inference, returning the output within 30 seconds for models up to 500MB in size.
2. THE Inference_Engine SHALL execute each inference task in an isolated process to prevent data leakage between concurrent inference requests.
3. WHEN inference execution completes successfully, THE Inference_Engine SHALL produce an Execution_Receipt containing the inference output, model identifier, input hash, output hash, and execution timestamp.
4. IF the specified model identifier does not exist in the Model_Registry, THEN THE Inference_Engine SHALL return an error response with a `ModelNotFound` status code.
5. IF inference execution exceeds the configured timeout (30 seconds), THEN THE Inference_Engine SHALL terminate the execution and return an error response with a `TimeoutExceeded` status code.

6. THE Inference_Engine SHALL log each inference execution with a unique execution identifier, model identifier, input hash, and execution duration for audit purposes.

Requirement 2: Trusted Execution Environment Integration

User Story: As a platform operator, I want AI inference to execute within a Trusted Execution Environment, so that model integrity and data confidentiality are hardware-attested and tamper-proof.

Acceptance Criteria

1. WHEN the TEE_Runtime initializes, THE TEE_Runtime SHALL establish a hardware-attested enclave with encrypted memory isolation before accepting inference workloads.
2. WHEN a model is loaded into the TEE_Runtime, THE TEE_Runtime SHALL verify the model binary integrity by comparing its hash against the registered hash in the Model_Registry.
3. WHILE inference is executing within the TEE_Runtime, THE TEE_Runtime SHALL maintain memory encryption and prevent external processes from reading enclave memory.
4. WHEN inference execution completes within the TEE_Runtime, THE TEE_Runtime SHALL generate an Attestation_Report containing the enclave measurement, signer identity, input hash, output hash, and a hardware-bound signature.
5. IF the TEE hardware is unavailable or attestation generation fails, THEN THE TEE_Runtime SHALL reject the inference request and return an error response with a `TEEUnavailable` status code.
6. THE TEE_Runtime SHALL support both Intel SGX and AMD SEV simulation modes, selectable via configuration.

Requirement 3: Zero-Knowledge Machine Learning Proof Generation

User Story: As a verifier, I want cryptographic proofs that a specific ML model produced a specific output for a given input, so that I can verify inference correctness without accessing model weights.

Acceptance Criteria

1. WHEN the Proof_Generator receives an Execution_Receipt from the Inference_Engine, THE zkML_Prover SHALL generate a zero-knowledge proof attesting that the declared model produced the declared output for the declared input.
2. THE zkML_Prover SHALL produce proofs that are verifiable in constant time regardless of model size.

3. WHEN proof generation completes, THE zkML_Prover SHALL return a proof artifact containing the proof bytes, public inputs (input hash, output hash, model hash), and a verification key reference.
4. IF proof generation fails due to an unsupported model architecture, THEN THE zkML_Prover SHALL return an error response specifying the supported model architectures.
5. THE zkML_Prover SHALL generate proofs within 120 seconds for supported model architectures up to 500MB in parameter size.
6. THE zkML_Prover SHALL produce proofs that do not reveal model weights, training data, or intermediate activations to the verifier.

Requirement 4: Federated Learning Pipeline Orchestration

User Story: As a data scientist, I want to orchestrate federated learning across distributed participants, so that models can be trained collaboratively without centralizing sensitive data.

Acceptance Criteria

1. WHEN a federated learning task is initiated, THE Federated_Learning_Orchestrator SHALL create a Flower server instance and register participating clients based on the task configuration.
2. THE Federated_Learning_Orchestrator SHALL coordinate training rounds by distributing the current global model to registered clients and collecting model updates after each local training epoch.
3. WHEN all participating clients submit their model updates for a training round, THE Federated_Learning_Orchestrator SHALL aggregate the updates using the configured aggregation strategy (FedAvg by default) and produce a new global model.
4. WHEN all configured training rounds are complete, THE Federated_Learning_Orchestrator SHALL store the final aggregated model in the Model_Registry and generate a CID for the model artifact.
5. IF a participating client fails to submit an update within the configured round timeout, THEN THE Federated_Learning_Orchestrator SHALL proceed with aggregation using the updates from responsive clients.
6. THE Federated_Learning_Orchestrator SHALL record provenance metadata for each training round including participating client count, aggregation strategy, round duration, and resulting model hash.

Requirement 5: Blockchain Event Listening

User Story: As the verifiable AI execution system, I want to listen to Team 3a's blockchain events, so that AI inference tasks are automatically triggered when tasks are created and funded on-chain.

Acceptance Criteria

1. WHEN Team 3a's TaskManager contract emits a `TaskCreated` event, THE Event_Listener SHALL detect the event within 15 seconds and extract the task identifier, creator address, description hash, deadline, and bounty amount.
2. WHEN Team 3a's Escrow_Contract emits a `TaskFunded` event, THE Event_Listener SHALL detect the event within 15 seconds and extract the task identifier, funder address, and funded amount.
3. WHEN both a `TaskCreated` and `TaskFunded` event are detected for the same task identifier, THE Event_Listener SHALL dispatch an inference workload to the Inference_Engine with the task parameters.
4. IF the Event_Listener loses connection to the blockchain node, THEN THE Event_Listener SHALL attempt reconnection with exponential backoff (starting at 1 second, maximum 60 seconds) and resume event processing from the last confirmed block.
5. THE Event_Listener SHALL maintain a persistent record of the last processed block number to prevent duplicate event processing after restarts.
6. IF a detected task has a deadline that has already passed, THEN THE Event_Listener SHALL skip the task and log a warning with the task identifier and expired deadline.

Requirement 6: Execution Proof Submission to Blockchain

User Story: As a platform operator, I want execution proofs submitted on-chain after AI inference, so that task completion can be verified and settlement can proceed.

Acceptance Criteria

1. WHEN the Inference_Engine produces a valid Execution_Receipt with associated proofs, THE Proof_Submitter SHALL submit the Attestation_Report and zkML proof to Team 3a's smart contract within 60 seconds of inference completion.
2. WHEN submitting proofs on-chain, THE Proof_Submitter SHALL include the task identifier, execution proof hash, model CID, and result hash in the transaction payload.
3. WHEN proof submission is confirmed on-chain, THE Proof_Submitter SHALL emit a local event indicating successful proof anchoring with the transaction hash.
4. IF the on-chain proof submission transaction fails, THEN THE Proof_Submitter SHALL retry submission up to 3 times with a 10-second interval between retries.
5. IF all retry attempts for proof submission fail, THEN THE Proof_Submitter SHALL persist the failed submission to a dead-letter queue in Redis and emit an alert event for manual resolution.
6. THE Proof_Submitter SHALL batch multiple proof submissions into a single transaction when multiple inferences complete within a 10-second window to reduce gas costs.

Requirement 7: Content Identifier (CID) Management

User Story: As a verifier, I want model artifact CIDs recorded on-chain, so that the exact model version used for inference is permanently linked to the execution proof.

Acceptance Criteria

1. WHEN a model is loaded into the TEE_Runtime for inference, THE Inference_Engine SHALL resolve and record the CID of the loaded model binary from the Model_Registry.
2. WHEN a new model artifact is produced by the Federated_Learning_Orchestrator, THE Model_Registry SHALL upload the model binary to decentralized storage and store the returned CID.
3. WHEN the Proof_Submitter submits execution proofs on-chain, THE Proof_Submitter SHALL include the model CID in the submission payload.
4. THE Model_Registry SHALL maintain a mapping between model identifiers, version numbers, and their corresponding CIDs.
5. IF the decentralized storage network is unreachable during model upload, THEN THE Model_Registry SHALL queue the upload and retry with exponential backoff (starting at 5 seconds, maximum 300 seconds).
6. THE Model_Registry SHALL validate that a CID resolves to a valid model binary before allowing it to be referenced in inference requests.

Requirement 8: Secure Model Loading and Execution

User Story: As a model owner, I want my model loaded and executed securely within the TEE, so that model weights remain confidential and are protected from extraction.

Acceptance Criteria

1. WHEN the TEE_Runtime loads a model binary, THE TEE_Runtime SHALL decrypt the model inside the enclave using a sealed encryption key that is inaccessible outside the TEE boundary.
2. THE TEE_Runtime SHALL verify the integrity of the decrypted model by computing its hash and comparing against the expected hash stored in the Model_Registry.
3. WHILE a model is loaded in the TEE_Runtime, THE TEE_Runtime SHALL prevent model weights from being exported, copied, or read by processes outside the enclave.
4. WHEN the TEE_Runtime unloads a model after inference completion, THE TEE_Runtime SHALL securely erase the decrypted model weights from enclave memory.
5. IF a model binary fails integrity verification after decryption, THEN THE TEE_Runtime SHALL reject the model, securely erase the decrypted data, and return an error response with an `IntegrityCheckFailed` status code.
6. THE TEE_Runtime SHALL support loading models up to 500MB in size within the enclave memory constraints.

Requirement 9: Proof Verification Interface

User Story: As an external verifier, I want an API to verify execution proofs without requiring TEE hardware, so that anyone can independently confirm that AI inference was performed correctly.

Acceptance Criteria

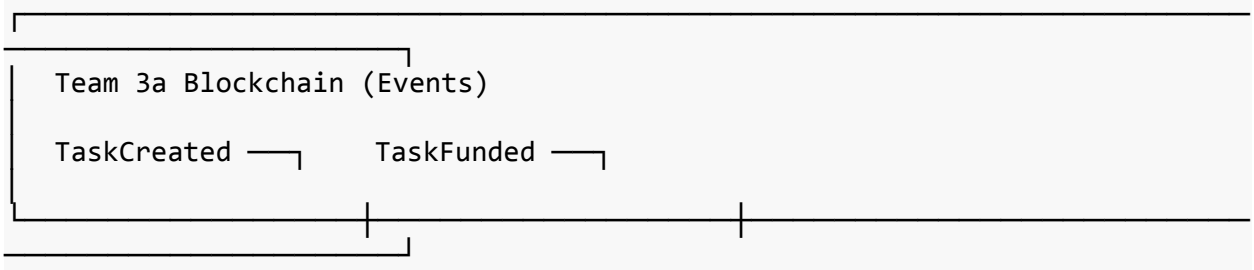
1. WHEN a verification request is submitted with a proof artifact and public inputs, THE Verification_Interface SHALL validate the proof and return a boolean verification result within 5 seconds.
2. THE Verification_Interface SHALL verify both TEE Attestation_Reports (validating the hardware signature chain) and zkML proofs (validating the computational correctness proof).
3. WHEN verification succeeds, THE Verification_Interface SHALL return a structured response containing the verification result (true), the verified public inputs, and the verification timestamp.
4. WHEN verification fails, THE Verification_Interface SHALL return a structured response containing the verification result (false) and a reason code indicating the failure cause (invalid_signature, expired_attestation, proof_mismatch, malformed_proof).
5. THE Verification_Interface SHALL accept proof artifacts by reference (on-chain transaction hash or CID) or by value (direct proof bytes in the request body).
6. IF the submitted proof format is unrecognized, THEN THE Verification_Interface SHALL return an error response specifying the supported proof formats (tee_attestation, zkml_snark).

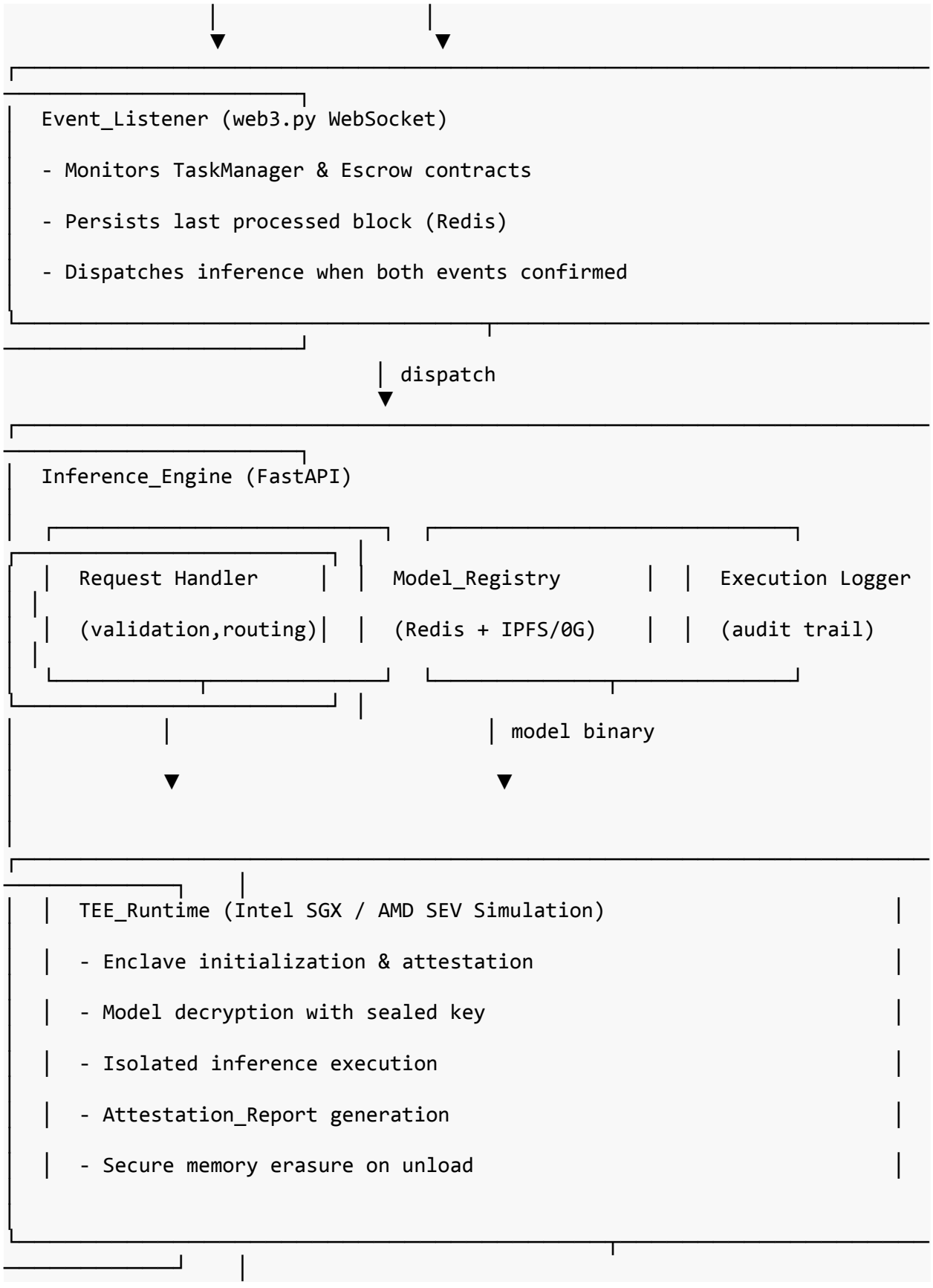
Design Document: Verifiable AI Execution (Team 3b)

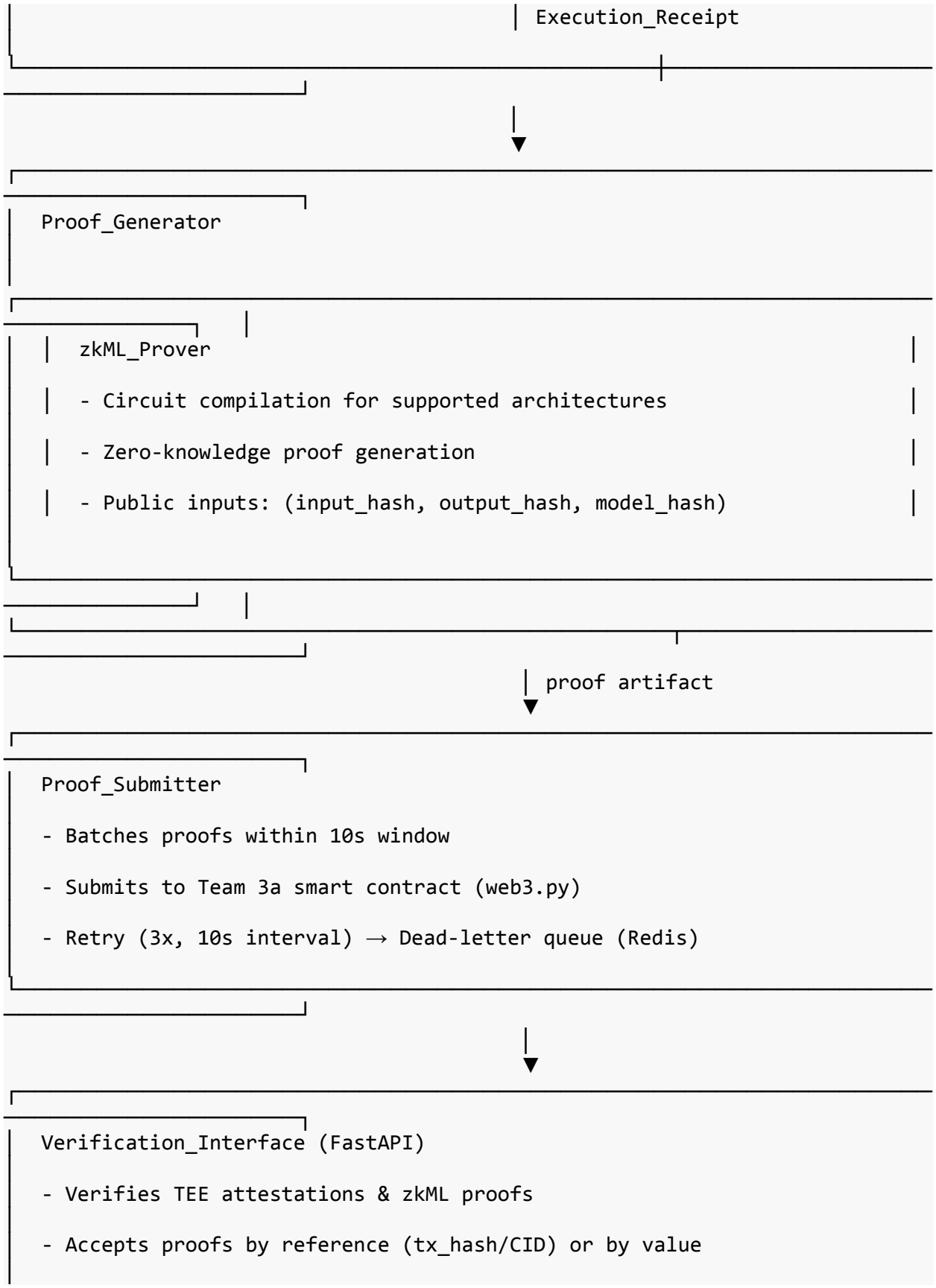
Introduction

This document defines the architecture and design for the Verifiable AI Execution subsystem. The system executes off-chain AI inference within Trusted Execution Environments (TEE), generates zero-knowledge proofs of correct execution (zkML), orchestrates federated learning via Flower, and submits cryptographic proofs on-chain for verification and settlement by Team 3a's smart contracts.

Architecture Overview




Event_Listener (web3.py WebSocket)
- Monitors TaskManager & Escrow contracts
- Persists last processed block (Redis)
- Dispatches inference when both events confirmed
dispatch
Inference_Engine (FastAPI)
Request Handler
(validation,routing)
Model_Registry
(Redis + IPFS/0G)
Execution Logger
(audit trail)
model binary
TEE_Runtime (Intel SGX / AMD SEV Simulation)
- Enclave initialization & attestation
- Model decryption with sealed key
- Isolated inference execution
- Attestation_Report generation
- Secure memory erasure on unload


Execution_Receipt
Proof_Generator
zkML_Prover
- Circuit compilation for supported architectures
- Zero-knowledge proof generation
- Public inputs: (input_hash, output_hash, model_hash)
proof artifact
Proof_Submitter
- Batches proofs within 10s window
- Submits to Team 3a smart contract (web3.py)
- Retry (3x, 10s interval) → Dead-letter queue (Redis)
Verification_Interface (FastAPI)
- Verifies TEE attestations & zkML proofs
- Accepts proofs by reference (tx_hash/CID) or by value

```
│   - Returns boolean + reason codes
│
└──────────────────────────────────────────────────────────────────────────────
──────────────────────────┘

┌──────────────────────────────────────────────────────────────────────────────
──────────────────────────┐
│   Federated_Learning_Orchestrator (Flower)
│
│   - Creates FL server per task
│
│   - Distributes global model, collects updates
│
│   - FedAvg aggregation (configurable)
│
│   - Stores final model → Model_Registry → CID
│
└──────────────────────────────────────────────────────────────────────────────
──────────────────────────┘
```

Components

1. Inference Engine

Responsibility: Receive inference requests, load models via TEE, execute inference, produce Execution_Receipts.

```python
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import hashlib
import time
import uuid
import asyncio

app = FastAPI()

class InferenceRequest(BaseModel):
    model_id: str
    input_data: bytes
    task_id: str | None = None

class ExecutionReceipt(BaseModel):
    execution_id: str
    model_id: str
    model_cid: str
    input_hash: str
    output_hash: str
    output: bytes
    timestamp: float
```

```python
class InferenceEngine:
    def __init__(self, model_registry, tee_runtime, timeout: float = 30.0):
        self.model_registry = model_registry
        self.tee_runtime = tee_runtime
        self.timeout = timeout

    async def execute(self, request: InferenceRequest) -> ExecutionReceipt:
        # Resolve model from registry
        model_entry = await self.model_registry.get(request.model_id)
        if model_entry is None:
            raise ModelNotFoundError(request.model_id)

        # Load model into TEE and execute with timeout
        try:
            output = await asyncio.wait_for(
                self.tee_runtime.execute_inference(
                    model_binary=model_entry.binary,
                    model_hash=model_entry.hash,
                    input_data=request.input_data
                ),
                timeout=self.timeout
            )
        except asyncio.TimeoutError:
            raise TimeoutExceededError(self.timeout)

        # Produce execution receipt
        execution_id = str(uuid.uuid4())
        receipt = ExecutionReceipt(
            execution_id=execution_id,
            model_id=request.model_id,
            model_cid=model_entry.cid,
            input_hash=hashlib.sha256(request.input_data).hexdigest(),
            output_hash=hashlib.sha256(output).hexdigest(),
            output=output,
            timestamp=time.time()
        )

        # Audit log
        await self._log_execution(receipt)
        return receipt

    async def _log_execution(self, receipt: ExecutionReceipt):
        """Persist audit log entry with execution_id, model_id, input_hash,
duration."""
        ...
```

Isolation: Each inference runs in a separate subprocess within the TEE enclave. The `TEE_Runtime.execute_inference` method spawns an isolated process per request.

2. TEE Runtime

Responsibility: Provide hardware-attested execution isolation, model integrity verification, attestation report generation, and secure memory management.

```
from enum import Enum
from dataclasses import dataclass

class TEEMode(Enum):
    INTEL_SGX = "sgx"
    AMD_SEV = "sev"

@dataclass
class AttestationReport:
    enclave_measurement: str
    signer_identity: str
    input_hash: str
    output_hash: str
    hardware_signature: bytes
    timestamp: float

class TEERuntime:
    def __init__(self, mode: TEEMode, sealed_key: bytes):
        self.mode = mode
        self.sealed_key = sealed_key
        self._initialized = False

    async def initialize(self) -> None:
        """Establish enclave with encrypted memory isolation."""
        self._enclave = await self._create_enclave(self.mode)
        self._initialized = True

    async def execute_inference(
        self, model_binary: bytes, model_hash: str, input_data: bytes
    ) -> bytes:
        if not self._initialized:
            raise TEEUnavailableError("TEE not initialized")

        # Decrypt model inside enclave
        decrypted_model = self._enclave.decrypt(model_binary,
self.sealed_key)

        # Verify integrity
        computed_hash = hashlib.sha256(decrypted_model).hexdigest()
        if computed_hash != model_hash:
            self._enclave.secure_erase(decrypted_model)
            raise IntegrityCheckFailedError(computed_hash, model_hash)

        # Execute inference in isolation
```

```
        try:
            output = await self._enclave.run_isolated(decrypted_model,
input_data)
        finally:
            # Secure erasure after completion
            self._enclave.secure_erase(decrypted_model)

        return output

    async def generate_attestation(
        self, input_data: bytes, output: bytes
    ) -> AttestationReport:
        """Generate hardware-bound attestation report."""
        if not self._initialized:
            raise TEEUnavailableError("Cannot generate attestation")

        return AttestationReport(
            enclave_measurement=self._enclave.measurement,
            signer_identity=self._enclave.signer_id,
            input_hash=hashlib.sha256(input_data).hexdigest(),
            output_hash=hashlib.sha256(output).hexdigest(),
            hardware_signature=self._enclave.sign(input_data + output),
            timestamp=time.time()
        )
```

Mode Selection: Configured via environment variable `TEE_MODE=sgx|sev`. The enclave implementation is abstracted behind `_create_enclave()` which returns a mode-specific adapter.

3. zkML Proof Generator

Responsibility: Generate zero-knowledge proofs that a specific model produced a specific output without revealing model internals.

```
from dataclasses import dataclass

SUPPORTED_ARCHITECTURES = ["linear", "cnn", "mlp", "transformer_small"]

@dataclass
class ProofArtifact:
    proof_bytes: bytes
    public_inputs: dict  # {input_hash, output_hash, model_hash}
    verification_key_ref: str

class ZkMLProver:
    def __init__(self, proving_key_store: str):
        self.proving_key_store = proving_key_store

    async def generate_proof(self, receipt: ExecutionReceipt) ->
```

```
ProofArtifact:
        """Generate zkML proof from execution receipt."""
        model_arch = await self._detect_architecture(receipt.model_id)
        if model_arch not in SUPPORTED_ARCHITECTURES:
            raise UnsupportedArchitectureError(model_arch,
SUPPORTED_ARCHITECTURES)

        # Compile circuit for model architecture
        circuit = await self._compile_circuit(model_arch, receipt.model_id)

        # Generate proof (witness: model weights + input, public: hashes)
        proof_bytes = await circuit.prove(
            private_inputs={
                "model_weights": receipt.model_id,  # resolved internally
                "input_data": receipt.input_hash,
            },
            public_inputs={
                "input_hash": receipt.input_hash,
                "output_hash": receipt.output_hash,
                "model_hash": receipt.model_cid,
            }
        )

        return ProofArtifact(
            proof_bytes=proof_bytes,
            public_inputs={
                "input_hash": receipt.input_hash,
                "output_hash": receipt.output_hash,
                "model_hash": receipt.model_cid,
            },
            verification_key_ref=circuit.vk_reference
        )
```

Zero-knowledge guarantee: The proof circuit takes model weights as private witness. The proof artifact contains only the succinct proof (constant-size ~256 bytes) and public input hashes. Model weights, intermediate activations, and training data are never exposed.

4. Federated Learning Orchestrator

Responsibility: Coordinate federated learning rounds using Flower framework, aggregate model updates, store trained models.

```
import flwr as fl
from flwr.server.strategy import FedAvg
from dataclasses import dataclass

@dataclass
class FLTaskConfig:
```

```
    task_id: str
    num_clients: int
    num_rounds: int
    aggregation_strategy: str = "fedavg"
    round_timeout: float = 300.0

@dataclass
class RoundProvenance:
    round_number: int
    participating_clients: int
    aggregation_strategy: str
    round_duration: float
    resulting_model_hash: str

class FederatedLearningOrchestrator:
    def __init__(self, model_registry, storage_backend):
        self.model_registry = model_registry
        self.storage_backend = storage_backend

    async def run_training(self, config: FLTaskConfig) -> str:
        """Execute FL training pipeline. Returns model CID."""
        strategy = self._build_strategy(config)

        # Start Flower server
        server = fl.server.Server(
            client_manager=fl.server.SimpleClientManager(),
            strategy=strategy
        )

        provenance_records: list[RoundProvenance] = []

        for round_num in range(config.num_rounds):
            round_start = time.time()

            # Distribute model and collect updates (with timeout for
stragglers)
            results = await self._execute_round(server, config.round_timeout)

            # Aggregate using configured strategy
            aggregated_weights = strategy.aggregate_fit(round_num, results,
[])

            provenance_records.append(RoundProvenance(
                round_number=round_num,
                participating_clients=len(results),
                aggregation_strategy=config.aggregation_strategy,
                round_duration=time.time() - round_start,
                resulting_model_hash=self._hash_weights(aggregated_weights),
            ))
```

```
        # Store final model and generate CID
        model_binary = self._serialize_weights(aggregated_weights)
        cid = await self.storage_backend.upload(model_binary)
        await self.model_registry.register(config.task_id, cid, model_binary)

        return cid

    def _build_strategy(self, config: FLTaskConfig):
        if config.aggregation_strategy == "fedavg":
            return FedAvg(
                min_fit_clients=1,  # Proceed even with partial responses
                min_available_clients=config.num_clients,
            )
        raise ValueError(f"Unknown strategy: {config.aggregation_strategy}")
```

Fault tolerance: If clients fail to respond within `round_timeout`, the orchestrator proceeds with available updates (Flower's `min_fit_clients=1`). Provenance records the actual participating count per round.

5. Event Listener

Responsibility: Monitor Team 3a blockchain events and dispatch inference workloads.

```
from web3 import Web3
import asyncio
import redis.asyncio as redis

class EventListener:
    def __init__(
        self,
        w3: Web3,
        task_manager_address: str,
        escrow_address: str,
        redis_client: redis.Redis,
        inference_engine,
    ):
        self.w3 = w3
        self.redis = redis_client
        self.inference_engine = inference_engine
        self._task_events: dict[str, dict] = {}  # task_id -> {created,
funded}
        self.RECONNECT_BASE = 1.0
        self.RECONNECT_MAX = 60.0

    async def start(self):
        """Begin event monitoring from last processed block."""
        last_block = await self._get_last_processed_block()
        while True:
            try:
```

```
                await self._process_events(from_block=last_block)
            except ConnectionError:
                await self._reconnect_with_backoff()

    async def _process_events(self, from_block: int):
        """Process TaskCreated and TaskFunded events."""
        # Subscribe to events via WebSocket
        task_created_filter =
self.task_manager.events.TaskCreated.create_filter(
            fromBlock=from_block
        )
        task_funded_filter = self.escrow.events.TaskFunded.create_filter(
            fromBlock=from_block
        )

        for event in task_created_filter.get_all_entries():
            task_id = event.args.taskId.hex()
            deadline = event.args.deadline

            # Skip expired tasks
            if deadline < time.time():
                logger.warning(f"Skipping expired task {task_id},
deadline={deadline}")
                continue

            self._task_events.setdefault(task_id, {})["created"] = event.args
            await self._try_dispatch(task_id)

        for event in task_funded_filter.get_all_entries():
            task_id = event.args.taskId.hex()
            self._task_events.setdefault(task_id, {})["funded"] = event.args
            await self._try_dispatch(task_id)

        await self._persist_block_number(event.blockNumber)

    async def _try_dispatch(self, task_id: str):
        """Dispatch inference only when both events are received."""
        events = self._task_events.get(task_id, {})
        if "created" in events and "funded" in events:
            await self.inference_engine.execute(InferenceRequest(
                model_id=events["created"].descriptionHash.hex(),
                input_data=b"",  # resolved from task description
                task_id=task_id,
            ))

    async def _reconnect_with_backoff(self):
        """Exponential backoff: 1s, 2s, 4s, ..., max 60s."""
        delay = self.RECONNECT_BASE
        while True:
```

```
            await asyncio.sleep(delay)
            try:
                await self.w3.provider.connect()
                return
            except ConnectionError:
                delay = min(delay * 2, self.RECONNECT_MAX)

    async def _get_last_processed_block(self) -> int:
        block = await self.redis.get("event_listener:last_block")
        return int(block) if block else 0

    async def _persist_block_number(self, block_number: int):
        await self.redis.set("event_listener:last_block", block_number)
```

6. Proof Submitter

Responsibility: Submit execution proofs on-chain with batching, retry logic, and dead-letter queue.

```
import asyncio
import redis.asyncio as redis
from web3 import Web3

class ProofSubmitter:
    BATCH_WINDOW = 10.0  # seconds
    MAX_RETRIES = 3
    RETRY_INTERVAL = 10.0  # seconds

    def __init__(self, w3: Web3, contract, redis_client: redis.Redis):
        self.w3 = w3
        self.contract = contract
        self.redis = redis_client
        self._pending_proofs: list[dict] = []
        self._batch_timer: asyncio.Task | None = None

    async def submit_proof(self, task_id: str, attestation:
AttestationReport,
                           zkml_proof: ProofArtifact, model_cid: str):
        """Queue proof for batched submission."""
        payload = {
            "task_id": task_id,
            "proof_hash": hashlib.sha256(
                zkml_proof.proof_bytes + attestation.hardware_signature
            ).hexdigest(),
            "model_cid": model_cid,
            "result_hash": attestation.output_hash,
            "attestation": attestation,
            "zkml_proof": zkml_proof,
        }
        self._pending_proofs.append(payload)
```

```
        # Start batch window timer if not already running
        if self._batch_timer is None:
            self._batch_timer =
asyncio.create_task(self._flush_after_window())

    async def _flush_after_window(self):
        """Wait for batch window, then submit all pending proofs."""
        await asyncio.sleep(self.BATCH_WINDOW)
        proofs = self._pending_proofs.copy()
        self._pending_proofs.clear()
        self._batch_timer = None
        await self._submit_batch(proofs)

    async def _submit_batch(self, proofs: list[dict]):
        """Submit batched proofs with retry logic."""
        for attempt in range(self.MAX_RETRIES):
            try:
                tx_hash = await self._send_transaction(proofs)
                # Emit success event
                await self._emit_anchoring_event(tx_hash, proofs)
                return
            except Exception:
                if attempt < self.MAX_RETRIES - 1:
                    await asyncio.sleep(self.RETRY_INTERVAL)

        # All retries exhausted — dead-letter queue
        await self._persist_to_dlq(proofs)
        await self._emit_alert(proofs)

    async def _send_transaction(self, proofs: list[dict]) -> str:
        """Build and submit on-chain transaction with proof payload."""
        tx = self.contract.functions.submitProofs(
            [p["task_id"] for p in proofs],
            [p["proof_hash"] for p in proofs],
            [p["model_cid"] for p in proofs],
            [p["result_hash"] for p in proofs],
        ).build_transaction({"from": self.account})
        signed = self.w3.eth.account.sign_transaction(tx, self.private_key)
        tx_hash = await
self.w3.eth.send_raw_transaction(signed.rawTransaction)
        await self.w3.eth.wait_for_transaction_receipt(tx_hash)
        return tx_hash.hex()

    async def _persist_to_dlq(self, proofs: list[dict]):
        """Store failed submissions in Redis dead-letter queue."""
        for proof in proofs:
            await self.redis.lpush("proof_submitter:dlq", json.dumps(proof))
```

```
    async def _emit_alert(self, proofs: list[dict]):
        """Emit alert event for manual resolution."""
        ...

    async def _emit_anchoring_event(self, tx_hash: str, proofs: list[dict]):
        """Emit local event confirming on-chain anchoring."""
        ...
```

7. Model Registry

Responsibility: Manage model storage, versioning, CID resolution, and integrity validation.

```
import redis.asyncio as redis
from dataclasses import dataclass

@dataclass
class ModelEntry:
    model_id: str
    version: int
    cid: str
    hash: str
    binary: bytes | None = None  # Loaded on demand

class ModelRegistry:
    UPLOAD_BACKOFF_BASE = 5.0
    UPLOAD_BACKOFF_MAX = 300.0

    def __init__(self, redis_client: redis.Redis, storage_backend):
        self.redis = redis_client
        self.storage = storage_backend

    async def get(self, model_id: str, version: int | None = None) ->
ModelEntry | None:
        """Resolve model by ID (latest version if version is None)."""
        key = f"model:{model_id}:latest" if version is None else
f"model:{model_id}:v{version}"
        data = await self.redis.hgetall(key)
        if not data:
            return None
        return ModelEntry(
            model_id=model_id,
            version=int(data[b"version"]),
            cid=data[b"cid"].decode(),
            hash=data[b"hash"].decode(),
        )

    async def register(self, model_id: str, cid: str, model_binary: bytes):
        """Register model with CID after upload. Validates CID resolves
```

```
correctly."""
        # Validate CID resolves to valid binary
        resolved = await self.storage.retrieve(cid)
        if hashlib.sha256(resolved).hexdigest() !=
hashlib.sha256(model_binary).hexdigest():
            raise IntegrityError("CID does not resolve to expected binary")

        version = await self._next_version(model_id)
        model_hash = hashlib.sha256(model_binary).hexdigest()

        # Store mapping
        key = f"model:{model_id}:v{version}"
        await self.redis.hset(key, mapping={
            "version": version,
            "cid": cid,
            "hash": model_hash,
        })
        await self.redis.set(f"model:{model_id}:latest", key)

    async def upload_model(self, model_id: str, model_binary: bytes) -> str:
        """Upload to decentralized storage with exponential backoff on
failure."""
        delay = self.UPLOAD_BACKOFF_BASE
        while True:
            try:
                cid = await self.storage.upload(model_binary)
                await self.register(model_id, cid, model_binary)
                return cid
            except StorageUnavailableError:
                await asyncio.sleep(delay)
                delay = min(delay * 2, self.UPLOAD_BACKOFF_MAX)
```

8. Verification Interface

Responsibility: Allow external parties to verify execution proofs without TEE hardware.

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from enum import Enum

class ProofFormat(Enum):
    TEE_ATTESTATION = "tee_attestation"
    ZKML_SNARK = "zkml_snark"

class VerificationRequest(BaseModel):
    proof_format: str
    proof_bytes: bytes | None = None  # By value
    proof_reference: str | None = None  # By reference (tx_hash or CID)
    public_inputs: dict | None = None
```

```python
class VerificationResponse(BaseModel):
    result: bool
    public_inputs: dict | None = None
    reason_code: str | None = None  # invalid_signature, expired_attestation,
proof_mismatch, malformed_proof
    timestamp: float

SUPPORTED_FORMATS = [f.value for f in ProofFormat]

class VerificationInterface:
    def __init__(self, tee_verifier, zkml_verifier, proof_resolver):
        self.tee_verifier = tee_verifier
        self.zkml_verifier = zkml_verifier
        self.proof_resolver = proof_resolver

    async def verify(self, request: VerificationRequest) ->
VerificationResponse:
        # Validate format
        if request.proof_format not in SUPPORTED_FORMATS:
            raise UnsupportedFormatError(request.proof_format,
SUPPORTED_FORMATS)

        # Resolve proof bytes (by reference or direct)
        proof_bytes = request.proof_bytes
        if proof_bytes is None and request.proof_reference:
            proof_bytes = await
self.proof_resolver.resolve(request.proof_reference)

        # Route to appropriate verifier
        if request.proof_format == ProofFormat.TEE_ATTESTATION.value:
            result, reason = await self.tee_verifier.verify(proof_bytes,
request.public_inputs)
        else:
            result, reason = await self.zkml_verifier.verify(proof_bytes,
request.public_inputs)

        return VerificationResponse(
            result=result,
            public_inputs=request.public_inputs if result else None,
            reason_code=reason if not result else None,
            timestamp=time.time()
        )
```

Data Models

Core Data Structures

Entity	Fields	Storage
InferenceRequest	model_id, input_data, task_id	In-memory (request lifecycle)
ExecutionReceipt	execution_id, model_id, model_cid, input_hash, output_hash, output, timestamp	Redis (audit log)
AttestationReport	enclave_measurement, signer_identity, input_hash, output_hash, hardware_signature, timestamp	Bundled with proof submission
ProofArtifact	proof_bytes, public_inputs, verification_key_ref	Submitted on-chain
ModelEntry	model_id, version, cid, hash	Redis (metadata) + IPFS/0G (binary)
FLTaskConfig	task_id, num_clients, num_rounds, aggregation_strategy, round_timeout	Redis
RoundProvenance	round_number, participating_clients, aggregation_strategy, round_duration, resulting_model_hash	Redis

Event Flow

```
TaskCreated + TaskFunded → Event_Listener → Inference_Engine → TEE_Runtime
    → ExecutionReceipt → Proof_Generator → ProofArtifact → Proof_Submitter
→ On-chain
```

Error Handling

Component	Error Condition	Response
Inference_Engine	Model not found	`ModelNotFound` (404)

Component	Error Condition	Response
Inference_Engine	Execution timeout	`TimeoutExceeded` (408)
TEE_Runtime	Hardware unavailable	`TEEUnavailable` (503)
TEE_Runtime	Hash mismatch after decryption	`IntegrityCheckFailed` (422)
zkML_Prover	Unsupported architecture	Error listing supported architectures (400)
Proof_Submitter	Transaction failure (all retries)	Persist to Redis DLQ, emit alert
Event_Listener	Connection lost	Exponential backoff (1s–60s)
Model_Registry	Storage unreachable	Exponential backoff (5s–300s)
Model_Registry	CID resolution fails	Reject model reference
Verification_Interface	Unrecognized proof format	Error listing supported formats (400)
Verification_Interface	Invalid proof	`{result: false, reason_code: "..."}`

Deployment Architecture

```yaml
# docker-compose.yml
services:
  inference-engine:
    build: ./inference
    environment:
      - TEE_MODE=sgx
      - REDIS_URL=redis://redis:6379
      - MODEL_STORAGE_URL=http://ipfs:5001
    ports: ["8000:8000"]

  event-listener:
    build: ./event_listener
    environment:
      - WEB3_WS_URL=ws://hardhat:8545
      - REDIS_URL=redis://redis:6379
    depends_on: [redis, inference-engine]
```

```yaml
proof-submitter:
  build: ./proof_submitter
  environment:
    - WEB3_RPC_URL=http://hardhat:8545
    - REDIS_URL=redis://redis:6379
  depends_on: [redis]

verification-api:
  build: ./verification
  ports: ["8001:8001"]

fl-orchestrator:
  build: ./federated_learning
  environment:
    - FLOWER_SERVER_PORT=8080
    - REDIS_URL=redis://redis:6379
  ports: ["8080:8080"]

redis:
  image: redis:7-alpine
  ports: ["6379:6379"]

ipfs:
  image: ipfs/kubo:latest
  ports: ["5001:5001", "8080:8080"]
```

Correctness Properties

A property is a characteristic or behavior that should hold true across all valid executions of a system — essentially, a formal statement about what the system should do. Properties serve as the bridge between human-readable specifications and machine-verifiable correctness guarantees.

Property 1: Execution Receipt completeness and hash correctness

For any valid inference request with an existing model, the produced Execution_Receipt SHALL contain a unique execution_id, the correct model_id, model_cid, input_hash equal to SHA-256 of the input data, output_hash equal to SHA-256 of the output, and a valid timestamp.

Validates: Requirements 1.3, 1.6

Property 2: Inference isolation (no cross-contamination)

For any two concurrent inference requests with different model_ids or input_data, the output of each request SHALL depend only on its own model and input, never on the other request's data.

Validates: Requirements 1.2

Property 3: Model not found error for invalid identifiers

For any model identifier that does not exist in the Model_Registry, the Inference_Engine SHALL return a ModelNotFound error and produce no Execution_Receipt.

Validates: Requirements 1.4

Property 4: TEE model integrity verification

For any model binary loaded into the TEE_Runtime, if the computed SHA-256 hash does not match the hash stored in the Model_Registry, the model SHALL be rejected with IntegrityCheckFailed and the decrypted data SHALL be securely erased.

Validates: Requirements 2.2, 8.2, 8.5

Property 5: Attestation Report completeness

For any inference that completes successfully within the TEE_Runtime, the generated Attestation_Report SHALL contain enclave_measurement, signer_identity, input_hash matching SHA-256 of input, output_hash matching SHA-256 of output, and a valid hardware-bound signature.

Validates: Requirements 2.4

Property 6: zkML proof generation round-trip verification

For any valid Execution_Receipt from a supported model architecture, the generated zkML proof SHALL be independently verifiable against the public inputs (input_hash, output_hash, model_hash) and verification SHALL return true.

Validates: Requirements 3.1, 3.3

Property 7: zkML proof zero-knowledge property

For any generated proof artifact, the proof_bytes size SHALL be constant regardless of model size, and the proof SHALL not contain extractable model weights or intermediate activations.

Validates: Requirements 3.6

Property 8: Unsupported architecture error includes supported list

For any model with an architecture not in the supported set, the zkML_Prover SHALL return an error that contains the complete list of supported architectures.

Validates: Requirements 3.4

Property 9: FedAvg aggregation correctness

For any set of N client model weight updates (w1, w2, …, wN) with equal data sizes, the FedAvg aggregated model SHALL have weights equal to (w1 + w2 + … + wN) / N for each parameter.

Validates: Requirements 4.3

Property 10: Federated learning fault tolerance

For any training round where K out of N clients respond (K >= 1), the aggregation SHALL proceed using only the K responsive client updates, and the round provenance SHALL record participating_clients = K.

Validates: Requirements 4.5, 4.6

Property 11: FL training completion produces stored model with CID

For any completed federated learning task, the final model SHALL be stored in the Model_Registry with a valid CID that resolves to the model binary.

Validates: Requirements 4.4, 7.2

Property 12: Event parsing extracts all fields correctly

For any TaskCreated event emitted on-chain, the Event_Listener SHALL extract task_id, creator_address, description_hash, deadline, and bounty_amount matching the on-chain event data. *For any* TaskFunded event, it SHALL extract task_id, funder_address, and funded_amount.

Validates: Requirements 5.1, 5.2

Property 13: Dispatch requires both events

For any task identifier, the Event_Listener SHALL dispatch an inference workload if and only if both a TaskCreated and TaskFunded event have been received for that task_id, and the task deadline has not passed.

Validates: Requirements 5.3, 5.6

Property 14: Block persistence prevents duplicate processing

For any restart of the Event_Listener, processing SHALL resume from the last persisted block number, and no event SHALL be processed more than once.

Validates: Requirements 5.5

Property 15: Exponential backoff bounds

For any sequence of N consecutive connection failures, the reconnection delay SHALL equal min(base * 2^(N-1), max_delay), where base and max_delay are the configured bounds (1s/60s for Event_Listener, 5s/300s for Model_Registry).

Validates: Requirements 5.4, 7.5

Property 16: Proof submission payload completeness

For any proof submission transaction, the payload SHALL contain the task_id, execution_proof_hash, model_cid, and result_hash for every proof in the batch.

Validates: Requirements 6.2, 7.3

Property 17: Proof batching within time window

For any set of N proofs that complete within a 10-second window, the Proof_Submitter SHALL submit them in a single on-chain transaction rather than N separate transactions.

Validates: Requirements 6.6

Property 18: Retry exhaustion leads to dead-letter queue

For any proof submission that fails 3 consecutive times, the failed submission SHALL be persisted to the Redis dead-letter queue and an alert event SHALL be emitted.

Validates: Requirements 6.4, 6.5

Property 19: Model Registry round-trip

For any model registered with a model_id, version, and CID, querying the registry with that model_id and version SHALL return the exact CID that was registered.

Validates: Requirements 7.4

Property 20: CID validation before inference reference

For any CID referenced for inference, the Model_Registry SHALL verify that the CID resolves to a valid model binary before allowing inference to proceed.

Validates: Requirements 7.6

Property 21: Verification correctness for valid and invalid proofs

For any valid proof artifact with matching public inputs, the Verification_Interface SHALL return result=true with the verified public inputs and timestamp. *For any* invalid or tampered proof, it SHALL return result=false with an appropriate reason_code from {invalid_signature, expired_attestation, proof_mismatch, malformed_proof}.

Validates: Requirements 9.1, 9.2, 9.3, 9.4

Property 22: Verification accepts both reference and value inputs

For any proof submitted by reference (tx_hash or CID) or by value (direct bytes), the Verification_Interface SHALL resolve the proof and perform verification identically.

Validates: Requirements 9.5

Property 23: Unrecognized format error includes supported formats

For any verification request with a proof_format not in {tee_attestation, zkml_snark}, the Verification_Interface SHALL return an error specifying the supported formats.

Validates: Requirements 9.6

Implementation Plan: Verifiable AI Execution (Team 3b)

Overview

Implement the off-chain AI inference subsystem with TEE-isolated execution, zkML proof generation, federated learning orchestration via Flower, blockchain event listening, and on-chain proof submission. The system uses Python (FastAPI, web3.py, Flower, Redis) with Docker-based deployment.

Tasks

- ☐ 1. Set up project structure, core interfaces, and shared data models
  - ☐ 1.1 Create directory structure and project scaffolding
    - Create top-level directories: `inference/`, `event_listener/`, `proof_submitter/`, `verification/`, `federated_learning/`, `common/`, `tests/`
    - Add `pyproject.toml` or `requirements.txt` with dependencies: fastapi, uvicorn, web3, redis, flwr, pydantic, pytest, hypothesis
    - Add `docker-compose.yml` with services for inference-engine, event-listener, proof-submitter, verification-api, fl-orchestrator, redis, ipfs
    - *Requirements: All (infrastructure)*
  - ☐ 1.2 Define core data models and shared types

- Create `common/models.py` with Pydantic models: `InferenceRequest`, `ExecutionReceipt`, `AttestationReport`, `ProofArtifact`, `ModelEntry`, `FLTaskConfig`, `RoundProvenance`
- Create `common/exceptions.py` with custom exception classes: `ModelNotFoundError`, `TimeoutExceededError`, `TEEUnavailableError`, `IntegrityCheckFailedError`, `UnsupportedArchitectureError`, `StorageUnavailableError`
- Create `common/constants.py` with shared constants: `SUPPORTED_ARCHITECTURES`, `BATCH_WINDOW`, `MAX_RETRIES`, `RETRY_INTERVAL`
- *Requirements: 1.3, 1.4, 1.5, 2.5, 3.3, 3.4*

☐ 1.3 Define abstract interfaces for pluggable components

- Create `common/interfaces.py` with abstract base classes: `StorageBackend` (upload/retrieve), `Enclave` (decrypt/run_isolated/secure_erase/sign), `ProofResolver` (resolve), `TEEVerifier` (verify), `ZkMLVerifier` (verify)
- *Requirements: 2.6, 9.5*

☐ 2. Implement Model Registry

- ☐ 2.1 Implement ModelRegistry class with Redis-backed storage
  - Create `common/model_registry.py` with `ModelRegistry` class
  - Implement `get(model_id, version)` — resolve model by ID, latest version if version is None
  - Implement `register(model_id, cid, model_binary)` — validate CID resolves correctly, store mapping in Redis
  - Implement `upload_model(model_id, model_binary)` — upload to decentralized storage with exponential backoff (5s base, 300s max)
  - Implement `_next_version(model_id)` — auto-increment version counter
  - *Requirements: 7.1, 7.2, 7.4, 7.5, 7.6*
- [ ]* 2.2 Write property tests for Model Registry
  - **Property 19: Model Registry round-trip** — registering and querying returns exact CID

        - **Property 20: CID validation before inference reference** — invalid CID rejected
        - **Property 15: Exponential backoff bounds** (Model Registry variant) — delay = min(5 * 2^(N-1), 300)
        - **Validates: Requirements 7.4, 7.5, 7.6**
- ☐ 3. Implement TEE Runtime
    - ☐ 3.1 Implement TEE Runtime with enclave simulation
        - Create `inference/tee_runtime.py` with `TEERuntime` class
        - Implement `initialize()` — establish enclave with encrypted memory isolation
        - Implement `execute_inference(model_binary, model_hash, input_data)` — decrypt model, verify integrity, run isolated inference, secure erase after completion
        - Implement `generate_attestation(input_data, output)` — generate hardware-bound `AttestationReport`
        - Implement `TEEMode` enum (SGX/SEV) with mode-specific adapter via `_create_enclave()`
        - Create `inference/enclave_sim.py` with simulated enclave for development (implements `Enclave` interface)
        - *Requirements: 2.1, 2.2, 2.3, 2.4, 2.5, 2.6, 8.1, 8.2, 8.3, 8.4, 8.5, 8.6*
    - [ ]* 3.2 Write property tests for TEE Runtime
        - **Property 4: TEE model integrity verification** — hash mismatch → IntegrityCheckFailed + secure erase
        - **Property 5: Attestation Report completeness** — all fields present and hashes match SHA-256 of inputs/outputs
        - **Validates: Requirements 2.2, 2.4, 8.2, 8.5**
- ☐ 4. Implement Inference Engine
    - ☐ 4.1 Implement InferenceEngine class
        - Create `inference/engine.py` with `InferenceEngine` class

        - Implement `execute(request)` — resolve model from registry, load into TEE, execute with configurable timeout (30s), produce `ExecutionReceipt`
        - Handle error cases: model not found, timeout exceeded
        - Implement `_log_execution(receipt)` — persist audit log with execution_id, model_id, input_hash, duration
        - *Requirements: 1.1, 1.2, 1.3, 1.4, 1.5, 1.6*
    - ☐ 4.2 Create FastAPI application with inference endpoint
        - Create `inference/app.py` with FastAPI app
        - Add `POST /inference` endpoint accepting `InferenceRequest`, returning `ExecutionReceipt`
        - Add error handlers mapping custom exceptions to HTTP status codes (404, 408, 503, 422)
        - Add health check endpoint `GET /health`
        - *Requirements: 1.1, 1.4, 1.5*
    - [ ]* 4.3 Write property tests for Inference Engine
        - **Property 1: Execution Receipt completeness and hash correctness** — all fields present, hashes match SHA-256
        - **Property 2: Inference isolation (no cross-contamination)** — concurrent requests produce independent outputs
        - **Property 3: Model not found error for invalid identifiers** — returns ModelNotFoundError, no receipt produced
        - **Validates: Requirements 1.2, 1.3, 1.4, 1.6**
- ☐ 5. Checkpoint - Ensure all tests pass
    - Ensure all tests pass, ask the user if questions arise.
- ☐ 6. Implement zkML Proof Generator
    - ☐ 6.1 Implement ZkMLProver class
        - Create `proof_submitter/zkml_prover.py` with `ZkMLProver` class
        - Implement `generate_proof(receipt)` — detect model architecture, compile circuit, generate proof with private/public inputs

- Implement `_detect_architecture(model_id)` — determine model architecture from registry metadata
- Implement `_compile_circuit(model_arch, model_id)` — compile proving circuit for supported architectures
- Return `ProofArtifact` with proof_bytes, public_inputs (input_hash, output_hash, model_hash), verification_key_ref
- Handle unsupported architecture with error listing supported set: ["linear", "cnn", "mlp", "transformer_small"]
- *Requirements: 3.1, 3.2, 3.3, 3.4, 3.5, 3.6*

- [ ]* 6.2 Write property tests for zkML Prover
  - **Property 6: zkML proof generation round-trip verification** — generated proof verifies against public inputs
  - **Property 7: zkML proof zero-knowledge property** — proof_bytes constant size, no extractable weights
  - **Property 8: Unsupported architecture error includes supported list** — error contains complete supported list
  - **Validates: Requirements 3.1, 3.3, 3.4, 3.6**

☐ 7. Implement Proof Submitter

- ☐ 7.1 Implement ProofSubmitter class with batching and retry
  - Create `proof_submitter/submitter.py` with `ProofSubmitter` class
  - Implement `submit_proof(task_id, attestation, zkml_proof, model_cid)` — queue proof, start batch window timer
  - Implement `_flush_after_window()` — wait 10s batch window, then submit all pending proofs
  - Implement `_submit_batch(proofs)` — build and send on-chain transaction with retry (3x, 10s interval)
  - Implement `_persist_to_dlq(proofs)` — store failed submissions in Redis dead-letter queue
  - Implement `_emit_alert(proofs)` and `_emit_anchoring_event(tx_hash, proofs)` — emit events
  - *Requirements: 6.1, 6.2, 6.3, 6.4, 6.5, 6.6*

- [ ]* 7.2 Write property tests for Proof Submitter
  - **Property 16: Proof submission payload completeness** — payload contains task_id, proof_hash, model_cid, result_hash
  - **Property 17: Proof batching within time window** — proofs within 10s submitted in single transaction
  - **Property 18: Retry exhaustion leads to dead-letter queue** — 3 failures → DLQ + alert
  - **Validates: Requirements 6.2, 6.4, 6.5, 6.6**

☐ 8. Implement Event Listener

- ☐ 8.1 Implement EventListener class with blockchain monitoring
  - Create `event_listener/listener.py` with `EventListener` class
  - Implement `start()` — begin event monitoring from last processed block
  - Implement `_process_events(from_block)` — subscribe to TaskCreated and TaskFunded events via WebSocket
  - Implement `_try_dispatch(task_id)` — dispatch inference only when both events received and deadline not passed
  - Implement `_reconnect_with_backoff()` — exponential backoff (1s base, 60s max)
  - Implement `_get_last_processed_block()` / `_persist_block_number()` — Redis persistence for restart safety
  - Skip expired tasks (deadline < current time) and log warnings
  - *Requirements: 5.1, 5.2, 5.3, 5.4, 5.5, 5.6*
- [ ]* 8.2 Write property tests for Event Listener
  - **Property 12: Event parsing extracts all fields correctly** — TaskCreated and TaskFunded fields extracted
  - **Property 13: Dispatch requires both events** — dispatch iff both events received and deadline valid
  - **Property 14: Block persistence prevents duplicate processing** — restart resumes from last persisted block

        - **Property 15: Exponential backoff bounds** (Event Listener variant) — delay = min(1 * 2^(N-1), 60)
        - **Validates: Requirements 5.1, 5.2, 5.3, 5.4, 5.5, 5.6**
- ☐ 9. Checkpoint - Ensure all tests pass
    - Ensure all tests pass, ask the user if questions arise.
- ☐ 10. Implement Federated Learning Orchestrator
    - ☐ 10.1 Implement FederatedLearningOrchestrator class
        - Create `federated_learning/orchestrator.py` with `FederatedLearningOrchestrator` class
        - Implement `run_training(config)` — execute FL training pipeline using Flower, return model CID
        - Implement `_build_strategy(config)` — build FedAvg (or other configured) aggregation strategy
        - Implement `_execute_round(server, timeout)` — distribute model, collect updates with round timeout
        - Record `RoundProvenance` for each round (participating_clients, aggregation_strategy, round_duration, resulting_model_hash)
        - Store final model in Model_Registry and generate CID
        - *Requirements: 4.1, 4.2, 4.3, 4.4, 4.5, 4.6*
    - [ ]* 10.2 Write property tests for Federated Learning Orchestrator
        - **Property 9: FedAvg aggregation correctness** — aggregated weights = mean of client weights
        - **Property 10: Federated learning fault tolerance** — K out of N clients responding → proceed with K updates
        - **Property 11: FL training completion produces stored model with CID** — final model stored with valid CID
        - **Validates: Requirements 4.3, 4.4, 4.5, 4.6**
- ☐ 11. Implement Verification Interface
    - ☐ 11.1 Implement VerificationInterface class and API
        - Create `verification/verifier.py` with `VerificationInterface` class

        - Implement `verify(request)` — validate proof format, resolve proof bytes (by reference or value), route to appropriate verifier (TEE or zkML)
        - Return `VerificationResponse` with result, public_inputs (on success), reason_code (on failure), timestamp
        - Supported formats: `tee_attestation`, `zkml_snark`
        - Reason codes: `invalid_signature`, `expired_attestation`, `proof_mismatch`, `malformed_proof`
        - *Requirements: 9.1, 9.2, 9.3, 9.4, 9.5, 9.6*
    - ☐ 11.2 Create FastAPI application for verification endpoint
        - Create `verification/app.py` with FastAPI app
        - Add `POST /verify` endpoint accepting `VerificationRequest`, returning `VerificationResponse`
        - Add error handler for unsupported format returning supported formats list
        - Add health check endpoint `GET /health`
        - *Requirements: 9.1, 9.5, 9.6*
    - [ ]* 11.3 Write property tests for Verification Interface
        - **Property 21: Verification correctness for valid and invalid proofs** — valid → true, invalid → false with reason
        - **Property 22: Verification accepts both reference and value inputs** — both modes produce identical results
        - **Property 23: Unrecognized format error includes supported formats** — error lists supported formats
        - **Validates: Requirements 9.1, 9.2, 9.3, 9.4, 9.5, 9.6**
- ☐ 12. Wire components together and add integration pipeline
    - ☐ 12.1 Create end-to-end inference pipeline wiring
        - Create `inference/pipeline.py` that wires: EventListener → InferenceEngine → TEERuntime → ProofGenerator → ProofSubmitter

        - Implement factory functions to instantiate all components with proper dependencies
        - Create `common/config.py` with environment variable parsing for all service configuration (TEE_MODE, REDIS_URL, WEB3_WS_URL, MODEL_STORAGE_URL, etc.)
        - *Requirements: 1.1, 5.3, 6.1, 7.1*
    - ☐ 12.2 Create Dockerfiles and finalize docker-compose.yml
        - Create `inference/Dockerfile`, `event_listener/Dockerfile`, `proof_submitter/Dockerfile`, `verification/Dockerfile`, `federated_learning/Dockerfile`
        - Finalize `docker-compose.yml` with proper environment variables, ports, and dependency ordering
        - Add `.env.example` with required environment variables
        - *Requirements: All (deployment)*
    - [ ]* 12.3 Write integration tests for end-to-end pipeline
        - Test: event received → inference executed → proof generated → proof submitted
        - Test: FL training → model stored → model used for inference → proof generated
        - Test: verification of both TEE attestation and zkML proof formats
        - *Requirements: 1.1, 3.1, 5.3, 6.1, 9.1*
- ☐ 13. Final checkpoint - Ensure all tests pass
    - Ensure all tests pass, ask the user if questions arise.

Notes

- Tasks marked with * are optional and can be skipped for faster MVP
- Each task references specific requirements for traceability
- Checkpoints ensure incremental validation
- Property tests validate universal correctness properties from the design document
- Unit tests validate specific examples and edge cases
- TEE simulation mode is used for development; production would use actual SGX/SEV hardware

- The system depends on Team 3a's smart contracts for event emission and proof verification on-chain

Task Dependency Graph


```
{
  "waves": [
    { "id": 0, "tasks": ["1.1"] },
    { "id": 1, "tasks": ["1.2", "1.3"] },
    { "id": 2, "tasks": ["2.1", "3.1"] },
    { "id": 3, "tasks": ["2.2", "3.2", "4.1"] },
    { "id": 4, "tasks": ["4.2", "4.3", "6.1"] },
    { "id": 5, "tasks": ["6.2", "7.1", "8.1"] },
    { "id": 6, "tasks": ["7.2", "8.2", "10.1"] },
    { "id": 7, "tasks": ["10.2", "11.1"] },
    { "id": 8, "tasks": ["11.2", "11.3"] },
    { "id": 9, "tasks": ["12.1"] },
    { "id": 10, "tasks": ["12.2", "12.3"] }
  ]
}
```